

Predicting Goal Scoring in the NHL

Christopher Choi

The Problem

- NHL analytics lag behind other major sports
- Teams take thousands of shots per season (~3,500 per team)
- The goal is to build a predictive model that classifies shots into one of 2 categories:
Goal or No Goal
 - Excluded team and player-specific features, allowing the model to focus purely on the situational context of a shot
 - Quantify shot quality

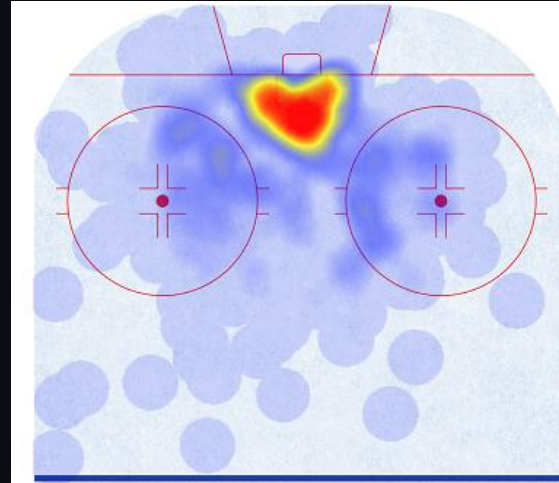


The NFL's NEXT GEN STATS logo

Note: Many of the decisions made when building this model were made in the interest of building technical skills and understanding different tools and techniques. While the final product lacks a clear business application, adjustments to the output of the model could be made to provide more practical applications.

Data

- Obtained from moneypuck.com
- Over 1 million shots recorded between 2015 and 2024, with over 100 features per shot
 - Extensive shot data related to players/teams, games, in-game state, and more.



Shooting Percentage: 11.2%
Percent of Shots On Net: 72.0%
Average Shot Distance: 21.6 feet
Expected Goals Per Shot: 0.103
Expected Goals Per Shot After Adjusting For Shooting Talent: 0.116
Percent of Shots That Generated Rebound Shots: 10.3%
Percent of Shots Frozen By Goalie: 12.4%

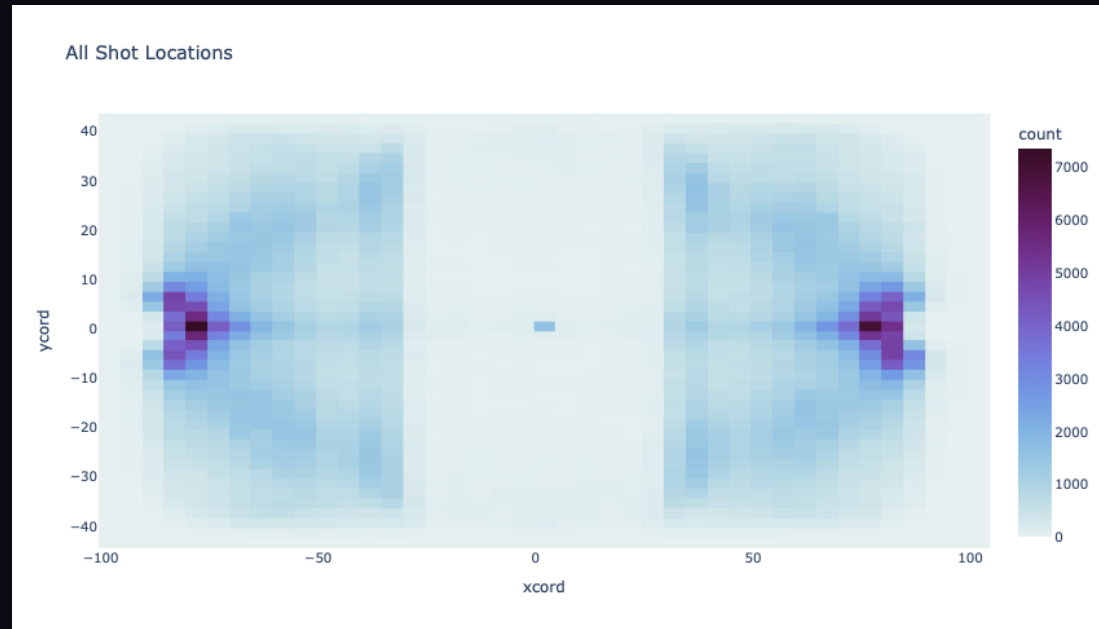
*Graphic showing the shots taken by Connor McDavid
in the 25-26 regular season*

Cleaning

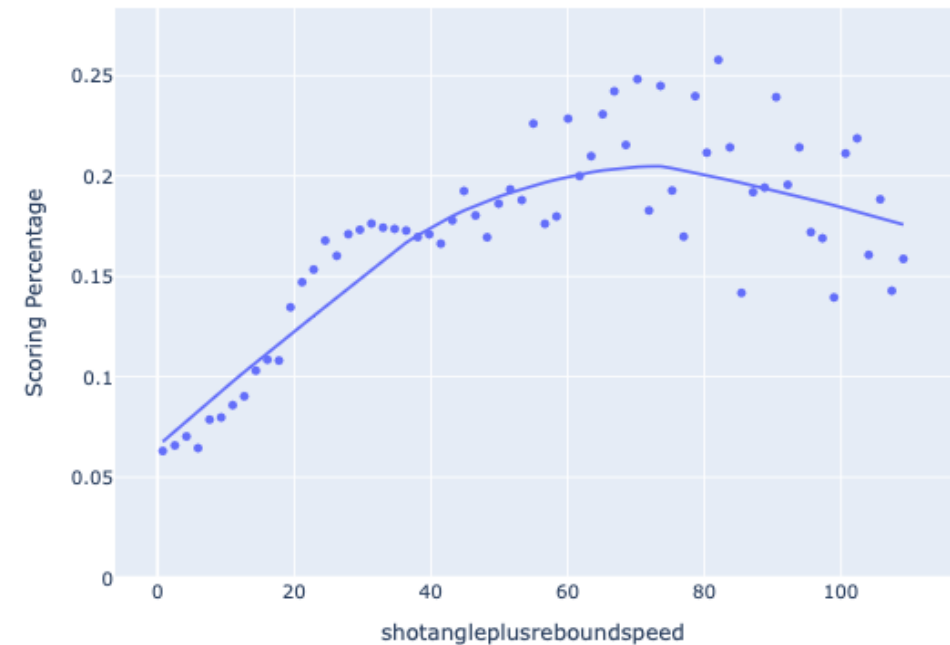
- The primary issue that I faced while cleaning this dataset was ensuring that there was no data leakage that could affect the model. Many of the features had to be dropped because they described the player/team taking the shot or contained information that could only be observed after the outcome of the shot had been determined.
- Grouping rare categories for encoding
- Handling missing/incorrect data
- Shift in direction
 - When I initially set out to clean the dataset, the end goal/scope of the project was different than where I ended up. As a result, a lot of the work I did ended up not being applicable
 - Creating a unique "Game Key" that allowed me to merge data across different datasets
 - Normalizing name spellings

EDA

In visualizing the data and getting a better sense of the dataset, I found that spatial, temporal, and contextual data all were significant when determining the likelihood of a shot resulting in a goal.



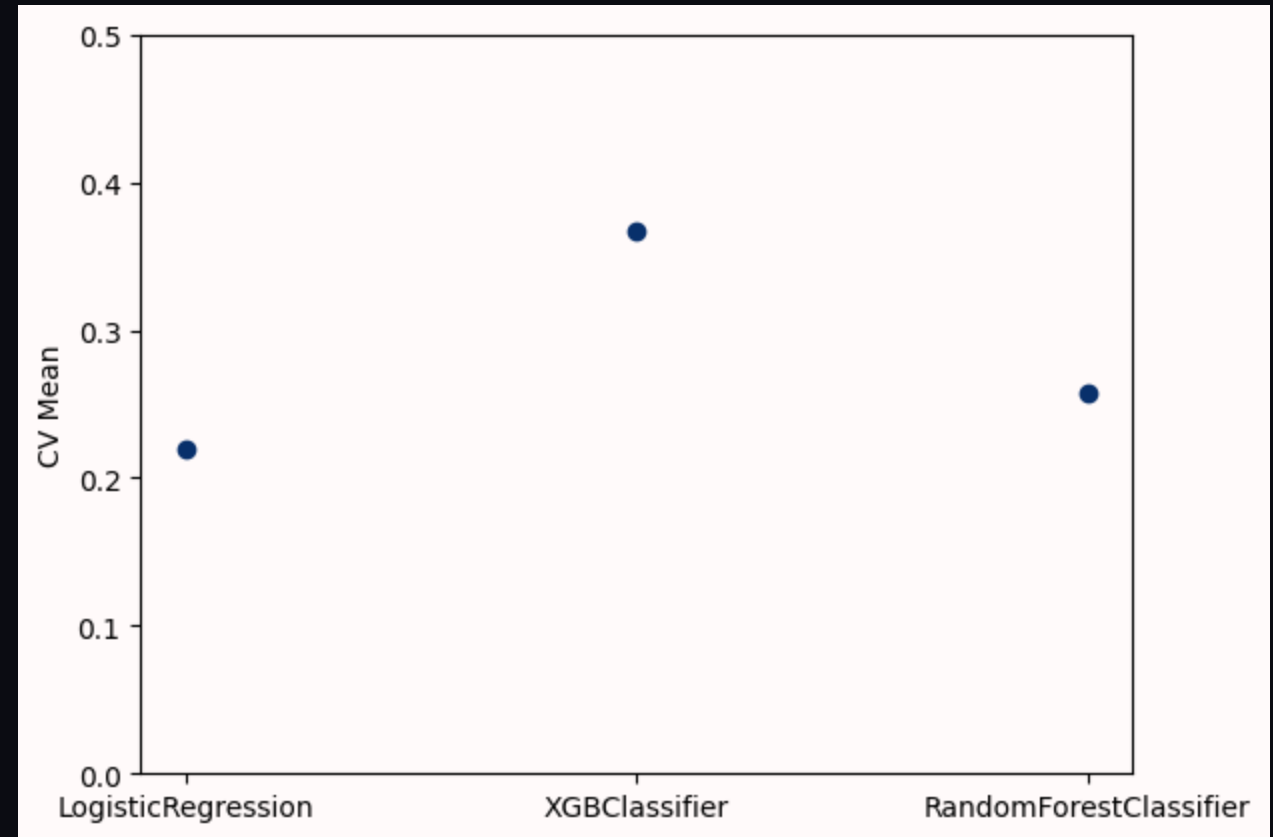
Scoring percentage given shotangleplusreboundsspeed



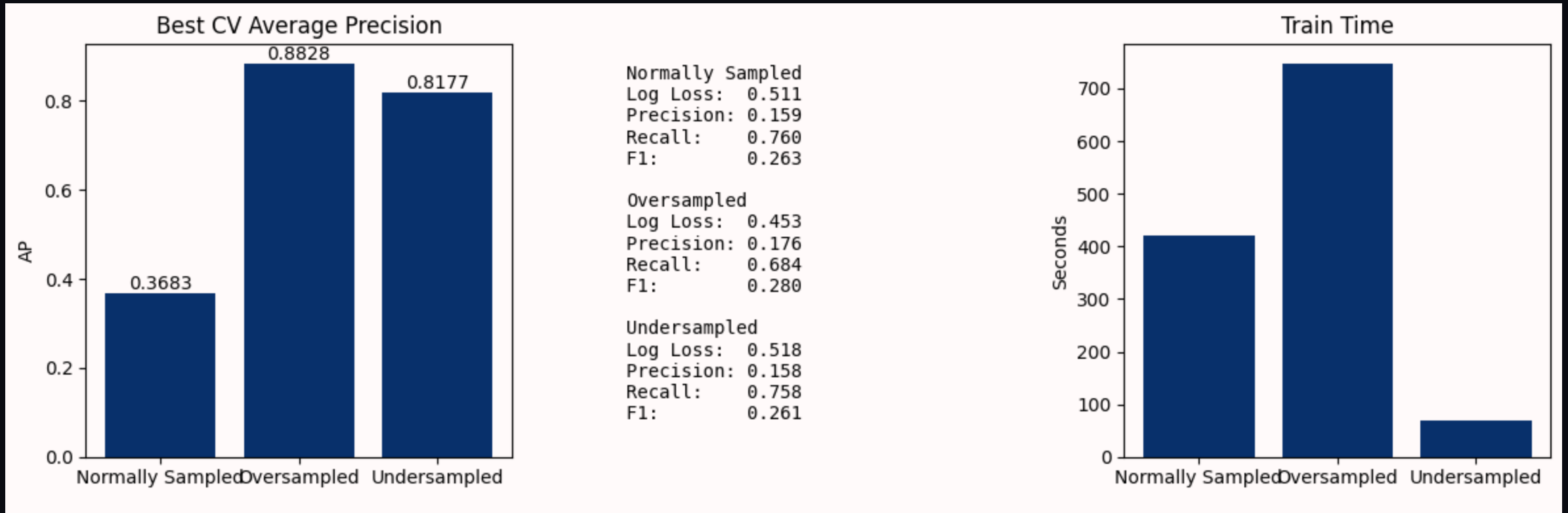
Shot angle plus rebound speed is an engineered feature that combines the change in angle from one shot to another, and the time between the two shots.

Modeling

- Tested 3 model types:
 - Logistic Regression
 - Random Forest (Bagging)
 - XGBoost (Boosting)
- Using PR AUC (AKA average precision) due to class imbalance, I found that boosting models performed best, which led me to test other boosting models



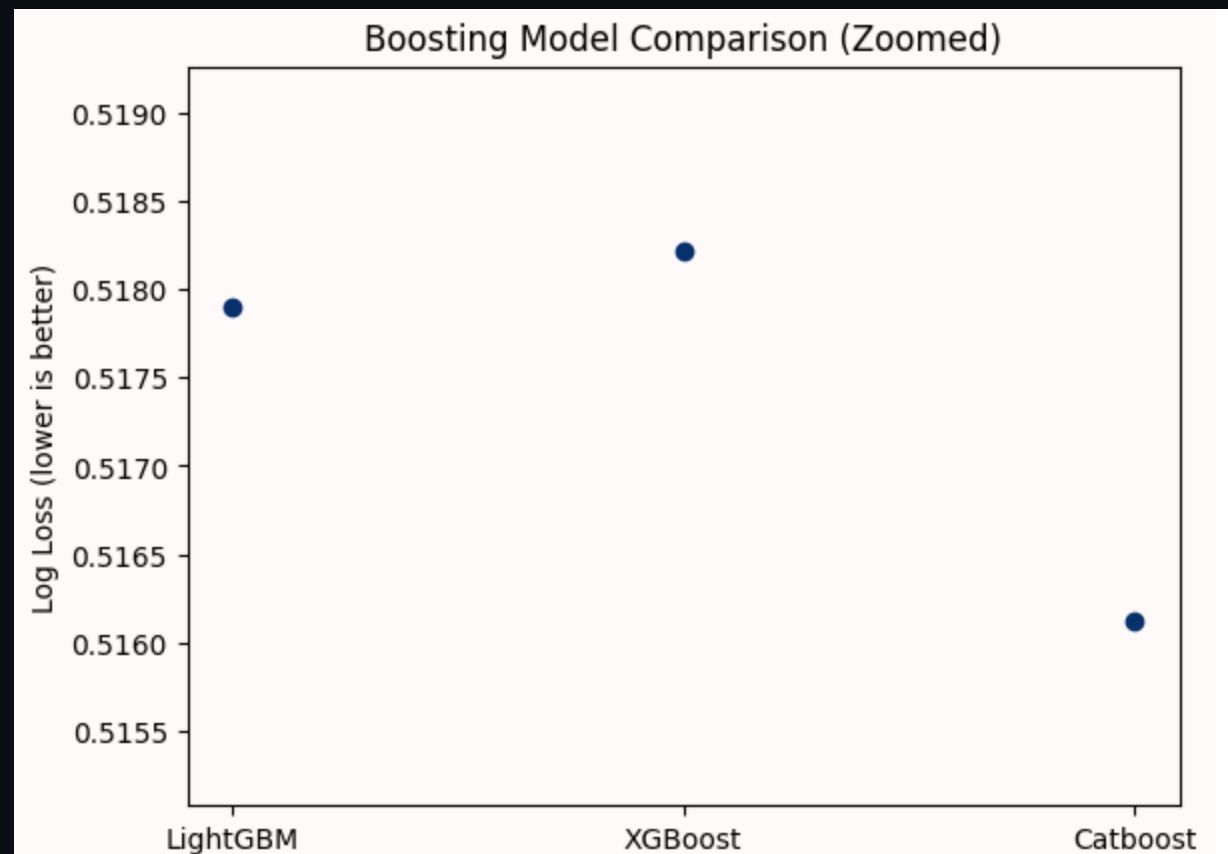
Handling Class Imbalance



Undersampling showed comparable performance and significantly improved performance

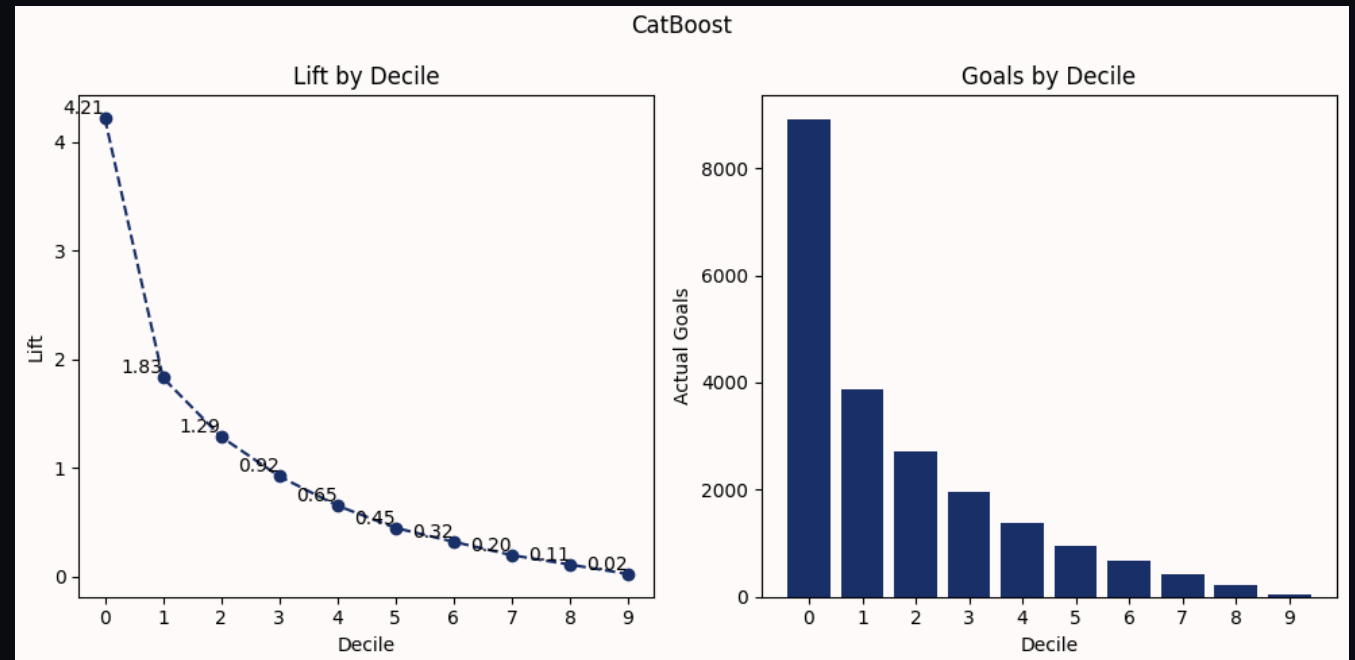
Boosting Models

- Switched to log loss as the primary performance metric (undersampling balanced the dataset, and CatBoost does not natively support average precision).
- XGBoost - Strong baseline that is well documented
- LightGBM - Computational efficiency and ability to scale to large datasets
- CatBoost - Handling of categorical variables



Decile Analysis / Threshold Tuning

- The number of false positives that the model was predicting at this point was far too high (low precision)
- In order to address this issue, I broke the shots into deciles and evaluated the probabilities relative to the baseline
 - The top 10% of shots were over 4.2x more likely to score than the baseline scoring rate
- Deciles 0-2 (top 30%) had more goals than the baseline rate, falling off to 0.92 in the third decile
- Demonstrates that the model successfully identifies high-danger scoring opportunities.



Conclusion

- Model provides foundation for:
 - Expected goals
 - Player evaluation
 - Strategy optimization
- Final confusion matrix with the threshold set to the top 30% of shots seen here:

